

Tasks 12 to 19: 5-Bus Network Simulator

FBS power flow for one time step, how it was checked, and the results

1. Overview

The network simulator was implemented using two main modules, `network_model.py` and `fbs.py`. `network_model.py` stores the distribution network and its connections, while `fbs.py` calculates the bus voltages, branch currents, power flow, and line losses. Separating these modules allows the network data to be changed without changing the power-flow calculation. The completed implementation calculates one operating point using a balanced radial network and constant-power loads.

A third short script, `run_simulation.py`, deals with the load and PV input and the hourly loop. It scales the loads and the PV output for each hour, calls the same solver, runs the checks and writes the results to a CSV file. The solver itself never knows about time. It only sees one snapshot at a time.

2. The 5-bus development case

The case is a small radial feeder on a 1 MVA, 12.66 kV base ($Z_{\text{base}} = 160.3 \text{ ohm}$). Bus 1 is the substation and is the slack bus, fixed at 1.0 p.u. Bus 2 feeds bus 3 and bus 5, and bus 3 feeds bus 4. Bus 4 also has a 180 kW PV unit running at unity power factor.

Line	R (p.u.)	X (p.u.)	Bus	Load P (kW)	Load Q (kVar)	PV (kW)
1-2	0.0150	0.0300	2	250	120	-
2-3	0.0100	0.0200	3	300	150	-
2-5	0.0180	0.0350	4	200	100	180
3-4	0.0120	0.0250	5	150	80	-

I did not start with this network. The first idea was the standard PJM 5-bus system, but it has six branches for five buses, so it has loops. FBS needs every bus except the slack to have exactly one line feeding it from upstream, so it cannot solve a looped network. That is why the case is a radial feeder, and why `network_model.py` has a `validate_radial()` check. If someone enters a looped network the code now stops with an error instead of giving numbers that look fine but are wrong.

PV is entered as a negative load at its bus. The net demand at a bus is the sum of its loads minus the PV output, so at bus 4 with full PV the real demand is $200 - 180 = 20 \text{ kW}$, while the reactive demand stays at 100 kVar because the PV has unity power factor.

3. Implementing `fbs.py` for one time step

`run_fbs(network, tol_pu, max_iter)` takes the network object and returns the bus voltages, branch currents, sending-end power flows, branch and total losses, the number of iterations and a converged flag. The steps inside are the standard ones:

First the buses are ordered by distance from the slack bus, so the code knows which buses are at the ends of the feeder. All voltages start at 1.0 p.u. at zero degrees. Each iteration then does a backward sweep, from the last bus back to the substation. For every bus the load current is $\text{conj}(S / V)$, and the current in the line feeding that bus is its own load current plus the currents of the lines leaving it. After that the forward sweep goes outward from the substation, and each bus voltage becomes its parent's voltage minus the line current times the line impedance. The loop ends when the biggest voltage change between two sweeps is below the tolerance ($1\text{e-}6$

p.u. was used), or after 100 iterations. Once it has converged, the sending-end power of each line is V times the conjugate of I , and the line loss is $|I|^2$ times Z .

```
for it in range(1, max_iter + 1):
    # backward sweep, leaves to root
    for b in reversed(bfs):
        if b == slack_id: continue
        i_load = (network.net_bus_injection_pu(b) / v[b]).conjugate()
        branch_current[b] = i_load + sum(branch_current[ln.to_bus] for ln in children[b])

    # forward sweep, root to leaves
    v_old = dict(v)
    for b in bfs:
        if b == slack_id: continue
        ln = parent_line[b]
        v[b] = v[ln.from_bus] - branch_current[b] * ln.z_pu

    if max(abs(v[b] - v_old[b]) for b in bfs) < tol_pu:
        converged = True
        break
```

Voltages and currents are kept as complex numbers all the way through, so the angles come out without any extra work. For the full-load, full-PV operating point the solver converged in 5 iterations.

4. Verifying the FBS result

A solver can give believable voltages and still be wrong, so the checks were done in a set order and each one is described below, including the ones that were only partly independent.

4.1 Hand-solved 3-bus case

Before using the 5-bus case, a 3-bus feeder was solved by hand (both lines $0.01 + j0.02$ p.u., loads $0.30 + j0.15$ and $0.20 + j0.10$ p.u., flat start). After the first sweep $|V2| = 0.99003$ and $|V3| = 0.98606$, after the second 0.98985 and 0.98578 , and it settled on the fourth iteration. The same 3-bus network was then put through fbs.py and gave the same answer, also in 4 iterations:

	V2 (p.u.)	V3 (p.u.)	Angle V2 (deg)	Angle V3 (deg)	Total loss (p.u.)
By hand	0.989843	0.985780	-0.4341	-0.6103	$0.003715 + j0.007429$
fbs.py	0.989843	0.985780	-0.4341	-0.6103	$0.003715 + j0.007429$

4.2 Power balance on the 5-bus case

The power that leaves the substation has to equal the loads plus the losses. The substation sends 733.963 kW into line 1-2. The net loads are $250 + 300 + 20 + 150 = 720.000$ kW and the losses are 13.963 kW, so $720.000 + 13.963 = 733.963$ kW. The mismatch is about 3×10^{-5} kW. It is not exactly zero because the iteration stops at a finite tolerance. The losses are 1.9% of the delivered load.

4.3 Expected-value checks

The voltage at bus 2 was estimated separately with the usual short-feeder approximation, $dV = (R P + X Q) / V$, using the power in line 1-2 ($0.734 + j0.478$ p.u.): $0.015 \times 0.734 + 0.030 \times 0.478 = 0.0253$, which gives $V2$ of about 0.9747. The solver gives 0.9748, which is close, as expected for a short line. The losses were also recomputed by hand from the branch currents as $I^2 R$, and they match the solver for every line (11.506, 1.769, 0.554 and 0.134 kW). I should say that this last check only tests the loss bookkeeping and not the currents themselves, because the currents come from the solver. The currents are what the power balance and the 3-bus comparison really test.

4.4 Tolerance trial

Tolerance (p.u.)	Iterations	V4 (p.u.)	Total loss (kW)
------------------	------------	-----------	-----------------

1e-3	3	0.96342244	13.962196
1e-6	5	0.96342032	13.963418
1e-9	7	0.96342032	13.963420
1e-12	9	0.96342032	13.963420

Tightening the tolerance below 1e-6 only adds iterations and changes nothing visible, so 1e-6 was kept. This also explains why the earlier Excel file says 7 iterations (it was run with 1e-9) while the later runs say 5.

4.5 Other things that were checked or fixed

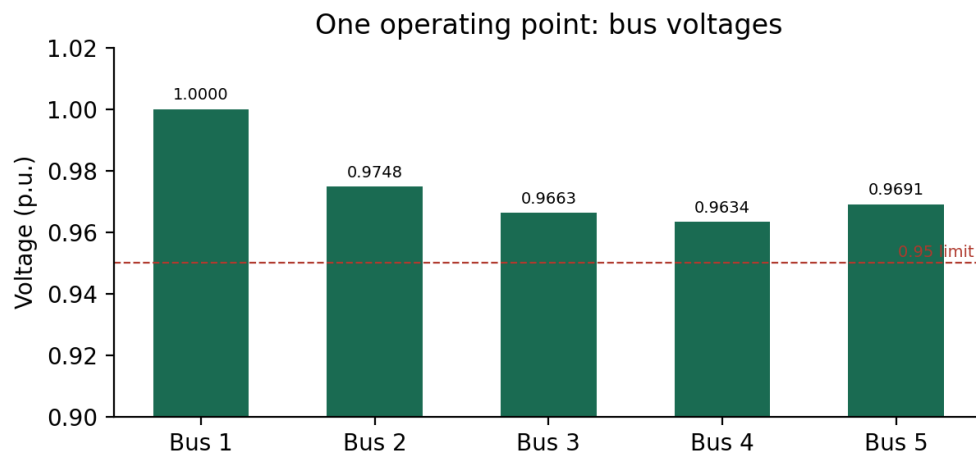
The PV sign was checked by looking for the 20 kW net demand at bus 4 in the results. Had it been added as load instead of subtracted, bus 4 would have shown 380 kW. The radial check was added after the PJM network failed, as described in section 2. When the working folder was reset during the project, the solver was run again from the saved files and gave the same numbers, so the results can be repeated.

5. Basic network results (full load, full PV)

Bus	V (p.u.)	Angle (deg)	Voltage (kV)
1 (slack)	1.00000	0.000	12.660
2	0.97477	-0.873	12.341
3	0.96626	-1.110	12.233
4	0.96342	-1.067	12.197
5	0.96908	-1.104	12.269

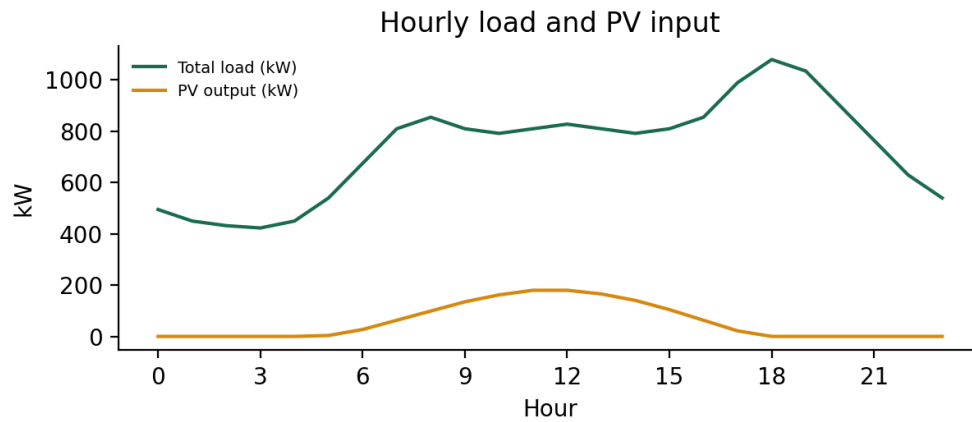
Line	(p.u.)	P sent (kW)	Q sent (kVAr)	S sent (MVA)	Loss (kW)	Loss (kVAr)
1-2	0.87584	733.96	477.91	0.8758	11.506	23.013
2-3	0.42054	321.90	253.82	0.4099	1.769	3.537
2-5	0.17542	150.55	81.08	0.1710	0.554	1.077
3-4	0.10585	20.13	100.28	0.1023	0.134	0.280

Total losses are 13.963 kW and 27.907 kVAr. The voltage falls steadily from the substation to the end of each branch, and bus 4 is the lowest because it is the furthest bus. All voltages are inside the 0.95 to 1.05 p.u. band in this case.



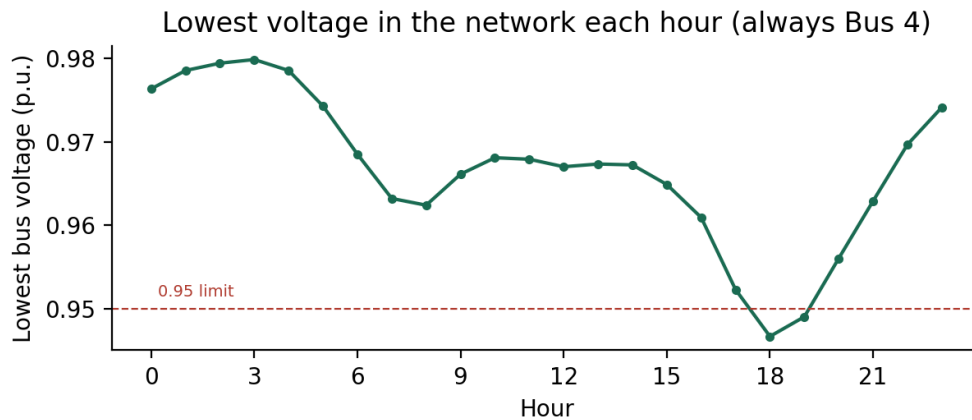
6. Hourly simulation loop

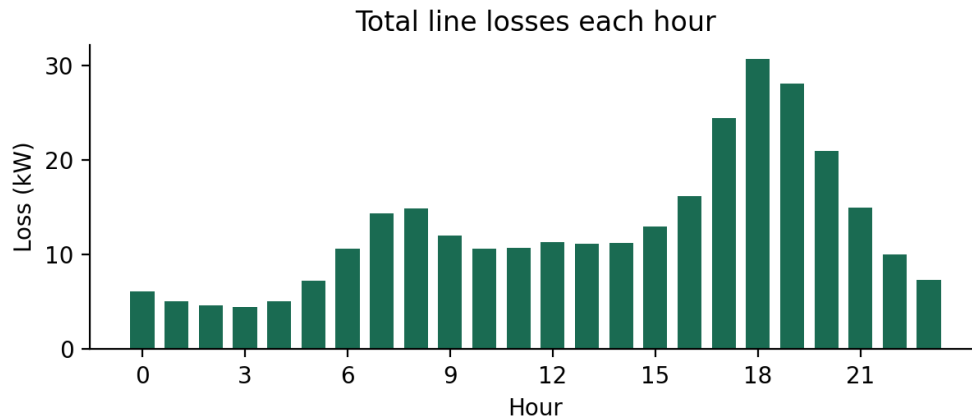
For the hourly run each hour has a load multiplier and a PV multiplier, which scale the base values before the solver is called. These 24-hour profiles are made-up development values (low load overnight, an evening peak, PV peaking at midday). They are not measured data and are only there to test the loop. They should be replaced with the real Jeddah load and irradiance data later.



Hour	Load (kW)	PV (kW)	Substation (kW)	Loss (kW)	Lowest V (p.u.)	Iter.
3	423	0	427.4	4.44	0.9799	4
8	855	99	770.9	14.90	0.9624	5
12	828	180	659.4	11.36	0.9670	5
18	1080	0	1110.7	30.69	0.9467	5
19	1035	0	1063.1	28.07	0.9490	5

Selected hours. All 24 hours are in hourly_results.csv.





All 24 hours converged, in 4 or 5 iterations each. The power balance check was repeated for every hour and the largest mismatch was $3.3\text{e-}04$ kW. Over the day the substation supplies about 16,538 kWh and the line losses are about 305 kWh, which is 1.85%.

The lowest voltage is at bus 4 in every hour. At 3 am the load is lowest (423 kW) and the voltage is 0.9799 p.u. At midday the PV supplies most of bus 4's demand and the lowest voltage is about 0.967 p.u. The worst case is 6 pm: the load peaks at 1080 kW with no PV, losses reach 30.7 kW, and the lowest voltage drops to 0.9467 p.u., which is below the 0.95 limit. Hours 18 and 19 are both under the limit. So the development feeder has an evening under-voltage problem, which is something the battery could be used to help with later in the project.

7. Limitations

The model is a single-phase equivalent of a balanced network, so unbalance is not represented. Loads are constant power and do not change with voltage. There are no tap changers or capacitors, and no battery yet. The PV runs at unity power factor, and the hourly profiles are illustrative. None of these choices is built into fbs.py, so they can be changed in the network data or in the driver script without touching the solver.